IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 8 June 2025, accepted 15 June 2025, date of publication 24 June 2025, date of current version 1 July 2025.

*Digital Object Identifier 10.1109/ACCESS.2025.3582159*

RESEARCH ARTICLE

# Robust Image Steganography Approach Based on Edge Detection Combined With CNN Algorithm

**RANA AL-RAWASHDEH[1], MD MAHFUZUR RAHMAN[1,2], AND MAHMOOD NIAZI[1,2]**

[1]Department of Information and Computer Science, College of Computing and Mathematics, King Fahd University of Petroleum and Minerals, Dhahran 31261, Saudi Arabia

[2]Interdisciplinary Research Center for Intelligent Secure Systems (IRC-ISS), King Fahd University of Petroleum and Minerals, Dhahran 31261, Saudi Arabia

Corresponding author: Md Mahfuzur Rahman (mdmahfuzur.rahman@kfupm.edu.sa)

This work was supported by the Cybersecurity Research and the Innovation Pioneers Initiative provided by the National Cybersecurity Authority (NCA), Saudi Arabia, under Grant CRPG-25-2064.

**ABSTRACT** Image steganography conceals secret data within a cover image to generate a new image (stego image) in a manner that makes the secret data undetectable. The main problem in image steganography is to find the right balance between the maximum amount of secret data that can be transmitted or stored in the cover image and the quality of the stego image to make the secret data un-noticeable to human senses or other detection mechanisms. In this research, a new framework is proposed that integrates the edge detection strategy (using edge detectors) with the deep learning methods, such as a convolutional neural network (CNN), for making secret data embedding and extraction processes efficient. Firstly, the edges in the cover image are identified using a suitable edge detection method (i.e., using the canny or sobel algorithm), and then the secret data is embedded inside the edge-detected cover image using a deep learning approach, and finally, the created stego image is sent to the receiver. On the receiver side, the secret data is extracted from the stego image using the same deep learning model in a reverse manner. In this article, we considered three datasets, such as the Ting ImageNet, Bossbase, and USC-SIPI datasets, to make edge-detected cover images and then consider them to build a deep learning model. We then evaluated the performance of our proposed deep learning model based embedding and extraction approach using various metrics related to imperceptibility, capacity, and robustness. Experimental results show high imperceptibility with PSNR (Peak Signal-to-Noise Ratio) reaching up to 39.85 dB and SSIM (Structural Similarity Index) up to 0.997 when using sobel, and PSNR up to 33.08 dB and SSIM up to 0.995 when using canny. The developed model also provides a higher capacity of 8 bits per pixel (BPP) and have ensured the robustness of the system against common image processing attacks (e.g., JPEG compression, Gaussian noise, salt-and-pepper, speckle, and filtering) which is validated with PSNR values up to 76.2 dB and error rates (Re) ranging between 0.32 and 0.47. Security testing using state-of-the-art steganalyzers (Xu-Net and Ye-Net) shows detection accuracy around 0.45-0.67 only, indicating strong resistance to steganalysis.

**INDEX TERMS** Steganography, edge detection, deep learning.

## I. INTRODUCTION

The rise of the Cloud-based application, Internet of Things (IoT), digital commerce networks, and social media interactions, etc., has resulted in a large volume of data being created and shared [1]. As data transmitted over the Internet continues to grow, it has become increasingly important to prioritize data security. To protect the confidentiality, integrity, and accessibility of data, cryptography is commonly used. Cryptography involves encoding information in a manner that makes it incomprehensible to individuals [2]. On the other hand, data hiding (i.e., Steganography) differs from cryptography as it aims to conceal information in such a way that the potential attackers are unaware of the presence of the contents, making it appear natural and inconspicuous rather than making the contents unreadable [3], [4], [5], [6].

The associate editor coordinating the review of this manuscript and approving it for publication was Zhenhua Guo.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

Image steganography has three different types of images - the 'secret image' that is concealed and hidden within the another image, the 'cover image' is the image that conceals the secret image, and the 'stego image' that represents the image after merging the secret image within the cover image [7], [8]. While compared with cryptography and also while integrated with cryptography [9], steganography can be considered much stronger and safer, as the transmitted message hardly attracts any attention or suspicion from the attackers.

Most of the current steganography techniques available focus only on the embedding process without giving consideration to where exactly to hide the data or even without considering the content of the cover image [10]. A crucial aspect of image steganography is to enhance the capacity to accommodate more amounts of secret data within the cover image. But increasing the capacity should not compromise imperceptibility, where imperceptibility refers to the quality of the stego image after embedding the secret data so that the secret image within stego image should be undetectable or not noticeable [11]. In this situation, when more data is added to the image noticeable changes might occur, risking its imperceptibility. Conversely, if little data is hidden the stenographic system may not be effective for real world applications [12]. The problem is to find the right balance between capacity and imperceptibility to ensure the quality of the stego image. The conflict between these properties, however makes it even harder to solve the steganography problem, and this could be viewed as a form of complex problem to balance both properties. Our research aims to find the appropriate regions in the cover image that can accommodate the secret message in an efficient way. Previous researches suggest that hiding data in sharp areas (i.e., edges) of an image with contrasts makes it less noticeable, to the human eye, when compared to any uniform areas [13]. This motivates us to map the secret information in the appropriate edge regions of the cover image to create a quality stego image. So, placing secret messages in the edge area of images make the hidden data more challenging to detect as the edge area can accommodate more message bits and can better tolerate pixel value changes [14].

Our proposed framework firstly prepares a secret image and then embed the secret image in the appropriate location (edges) of the cover image using a deep learning model (e.g., CNN model [15]). To prepare the CNN model, canny [16] and sobel [17] - two efficient edge detectors, have been separately used to identify the edge areas of the images (which are later considered as cover images) in three different datasets. These edge-detected cover images, along with a set of secret images, have been used to build our deep learning model.

In this research, we answer the following Research Questions (RQ):

- RQ-1: Can embedding within edge areas improve the capacity and imperceptibility of stego image?
- RQ-2: Can deep learning method such as CNN for embedding achieve better results?
- RQ-3: Is it possible to achieve equilibrium state between capacity, imperceptibility, robustness, and security of image steganography?

The answer to this research questions leads us to develop a novel steganography framework for ensuring security and protecting confidential details of data from unauthorized access. So our objectives of this research can be summarized as follow:

- Objective 1: Design, develop, and implement a novel image steganography framework for hiding sensitive data on images with a deep learning approach based on edge detection features.
- Objective 2: Evaluate two edge detection algorithms to select the best to be used with the proposed steganography framework.
- Objective 3: Compare the performance of the proposed framework based on various embedding performance parameters (i.e., hiding capacity, imperceptibility, robustness, etc.) and also evaluate the security aspects against steganalysis.

Unlike traditional steganographic systems that embed secret data uniformly or randomly within cover images, our proposed method intelligently utilizes edge regions detected via canny or sobel filters, which are known to be visually and structurally complex. This design choice enhances imperceptibility and robustness by targeting areas where pixel-level alterations are less noticeable to both the human eye and statistical steganalysis tools. Our approach therefore, contributes a hybrid framework that bridges classical edge detection with modern deep learning, offering a novel pathway for high-capacity and secure data hiding. In summary, this study offers a novel, edge-aware steganographic framework powered by deep learning, addressing long-standing trade-offs between imperceptibility, capacity, and robustness, and paving the way for more intelligent data hiding systems.

The proposed method differs from previous steganographic methods because it uses a hybrid edge-guided CNN-based framework instead of focusing on spatial-domain LSB embedding or fixed region-based concealment. Again, the proposed method uses classical edge detection (canny and sobel) with a convolutional encoder-decoder architecture to enable targeted embedding in edge-rich regions, which improves both imperceptibility and robustness. The dual-edge inputs and adaptive channel concatenation in this approach make it a new contribution to the field.

The paper has been organized as follows. Section II covers the related work to this work. Section III covers the proposed methodology. Section IV discusses the experiment results and Section V presents the conclusion.

## II. RELATED WORK

The digital world's expanding requirement for secure hidden communication has made image steganography more prominent in recent times. The current steganographic methods,

which use spatial or frequency domain manipulation, face limitations in maintaining both imperceptibility and attack resistance. Researches now focus on content-adaptive and deep learning-based approaches, which are explored in this section in addition to the traditional approaches.

Alanzy et al. [18] offers a new steganography scheme that is based on the LSB (Least Significant Bit) approach and incorporates AES and Blowfish encryption to hide text into images. Their scheme solves the problems that exist in the conventional LSB methods by introducing randomness when selecting the pixels and by using a mask as the encryption key. Their developments aim at improving the encryption and compression algorithms, and the incorporation of machine learning and artificial intelligence to enhance the resistance against attacks. The proposed method uses frame based data hiding (by splitting an image into multiple frames), where the order of the sequential frames is also disrupted for increased resistance. Their work also optimizes the mask sizes and simplifies the encoding process through the use of faster encryption and decryption algorithms and the use of compression for large messages. Their findings resulted in the PSNR (peak signal to noise ratio) being equal to 85.1 and the MSE (mean square erro) being equal to 0.00018717.

Jebur et al. in [27] suggested an image steganography method where the secret key is first converted into a one dimensional bit stream array, and then this bit stream is XORed with the bit stream of the secret image. In this work, the LSB technique is used in two ways: in the first instance, only the least significant bit (or 1 bit of each byte) is used to hide the data, and this method is referred to as 1-LSB. On the second place, the four least significant bits of the right half-byte (or 4 bits of each byte) is used to hide the data, and this method is called 4-LSB. The assessment of each LSB process includes both subjective and objective evaluation. The subjective analysis is for both the human visual system (HVS), which comprises the eyes, optic nerves, and the brain's visual processing regions, while the objective analysis involves both PSNR and MSE metrics. The results show that the PSNR=29.3 dB and MSE=27.

Alabaichi et al. [28] proposed two methods based on least significant bit (LSB) and secret map using the 3D chaotic map. They bolster the security of these methods by integrating maps, specifically the 3D Chebyshev and 3D logistic maps. By leveraging these maps, they aim to elevate the security level in concealing data within images. The process entails embedding information while selecting pixels from a base image. Subsequently the proposed algorithm undergoes a thorough evaluation based on criteria to gauge its efficacy. These criteria encompass correlation coefficient, information entropy, homogeneity, contrast image histogram analysis, key sensitivity, concealment capacity, quality assessment index, MSE, PSNR, and image authenticity. Their findings of the assessment illustrate that their novel algorithm meets all specified criteria as it adeptly conceals data while maintaining the integrity of the concealed images. Moreover, this algorithm demonstrates resilience against attacks such as statistical attacks.

In [2], a deep learning-based steganography method is introduced to hide secret information in a cover image. For this, a convolutional neural network (CNN) with a deep supervision-based edge detector is used, which can consider more edge pixels than conventional edge detection algorithms. First, the cover image is processed by masking the last 5-bits of each pixel value. Then, an edge map is calculated to localize regions of high gradient magnitude or abrupt changes in pixel values, which are generally representative of edges or lines in a grayscale image. To identify the most important edge information, the gray-scale edge map is then thresholded to binary using both global and adaptive thresholding. The reason for applying different techniques for binarization is to prove the sensitivity of the edge detection method. The embedding rule of secret bits into the cover image is as follows: more bits into edge pixels and fewer bits into non-edge pixels. The experimental results on various standard images prove that the proposed method outperforms other state-of-the-art methods in terms of payload capacity.

Kich et al. [29] designed a general system of image steganography that is based on auto-encoding network architecture and the whole process is trained by end-to-end deep convolutional neural networks for both concealment and extraction. The network system consists of two sub-networks: one for hiding which is used by the sender to encode a color image of the same size. The other for extraction, which is used by the recipient to retrieve the secret image from the received stego image. The authors performed experiments on different types of images from a challenging images dataset which is publicly available, including ImageNet, CIFAR10, LFW, PASCAL-VOC12. The result indicates that the proposed method is generic in nature with respect to the type of images used and solves the problem of capacity at an acceptable PSNR and SSIM values. Habiban et al. [30] proposed two new image steganography techniques with a view to enhancing the image-embedding capacity. The methods will encrypt and conceal secret information in the edge region. They employed two hybrid methods for the edge detection of the images. The first method combines the Laplacian of Gaussian (LoG) with the wavelet transform algorithm, and the second method mixes the LOG and Canny. To assess the effectiveness of the proposed methods, different evaluation metrics were used, including embedding capacity, PSNR, MSE, and SSIM. The results show that the proposed method has a higher embedding capacity than the other methods, while imperceptibility in the cover image is still high.

From Table 1, we note that the gap in the previous work is in finding the right balance between the capacity, imperceptibility, security, and robustness. So in our work, we tried to solve this trade-off as much as possible.

**TABLE 1.** Summary of related works.

| Ref | Year | Method | Assessment Tools | Dataset |
|---|---|---|---|---|
| [19] | 2017 | LSB+ Edge | PSNR>50<br>BPP=1<br>Security=RS Analysis<br>Robustness=N/A | USC-SIPI/Standard image test |
| [20] | 2017 | Adaptive LSB | Avg PSNR=45 dB<br>Avg SSIM=0.99<br>PC=8KB<br>Security=N/A<br>Robustness=Salt and Pepper | USC-SIPI/Standard image test |
| [21] | 2018 | LSB+ Location Map | Avg PSNR=51 dB<br>BPP=1.9<br>Security=RS analysis<br>Robustness=N/A | USC-SIPI/Standard image test |
| [22] | 2019 | LSB+ Edge | Avg PSNR=48 dB<br>Avg SSIM=0.99<br>PC=1835 bits<br>Security=Li-110D<br>Robustness=N/A | USC-SIPI/Standard image test |
| [23] | 2021 | LSB+GA | Avg PSNR=78 dB<br>BPP=1<br>Security=N/A<br>Robustness=N/A | USC-SIPI/Standard image test |
| [14] | 2018 | LSB+ Hybrid Edge | Avg PSNR=33 dB<br>BPP=3.3<br>Security=N/A<br>Robustness=N/A | USC-SIPI/Standard image test |
| [24] | 2022 | IWT | Avg PSNR=59 dB<br>Avg SSIM=0.85<br>PC=10000 bits<br>Security=Histogram<br>Robustness=N/A | DICOM Medical Images |
| [25] | 2022 | DCT+RS encoding+ STC | Avg PSNR=42 dB<br>Avg SSIM=0.99<br>PC=0.10 pnaz<br>Security=CCJRM<br>Robustness=Error rate | USC-SIPI/Standard image test |
| [26] | 2022 | Scalable DE | Avg PSNR=45 dB<br>PC=20kb<br>Security=N/A<br>Robustness=N/A | USC-SIPI/Standard image test |

## III. PROPOSED APPROACH

In this paper, we suggested a new approach that integrates edge regions detectors with CNN for embedding and extraction processes in image steganography. Firstly, we detect the edge areas using canny and sobel edge detection algorithms, as shown in Figure 1. Then the CNN is used for embedding a secret image (obtained from the secret message) within edge regions in the cover image to generate the stego image. The stego image can be sent across an insecure channel or stored in a storage system. To get the secret data back from the stego image, we pass the stego image through the CNN system, but in a reverse fashion.

### A. DATASETS ACQUISITION PROCESS AND PREPARATION

In this work, we test three datasets. The first dataset is the Tiny ImageNet dataset [31] that includes one hundred thousand (100,000) images divided into two hundred (200) classes, where each class contains five hundred (500) images resized to 64 × 64 colored pixels. Every class has five hundred (500) training images, fifty validations (50) images, and fifty (50) test images. The second dataset is the USC-SIPI image database [10] which is a collection of digitized images. The database is divided into volumes based on the nature of the images. The images in each volume are of different sizes, such as 256 × 256 pixels, 512 × 512 pixels, or 1024 × 1024 pixels. The third dataset is the Bossbase dataset [10], which comprises 10,000 grayscale images of size 512 × 512 and in the portable gray map (PGM) format. The images were captured with seven different cameras and then were converted to JPG format.

The selected datasets BOSSBase, Tiny ImageNet, and USC-SIPI were chosen to cover a wide range of image characteristics relevant to steganography. BOSSBase provides high-resolution natural grayscale images, useful for testing imperceptibility in real-world scenarios. Tiny ImageNet includes low-resolution color images with high semantic variation, which helps assess the model's generalization across diverse visual content. USC-SIPI offers synthetic and structured images with controlled edge and texture properties,

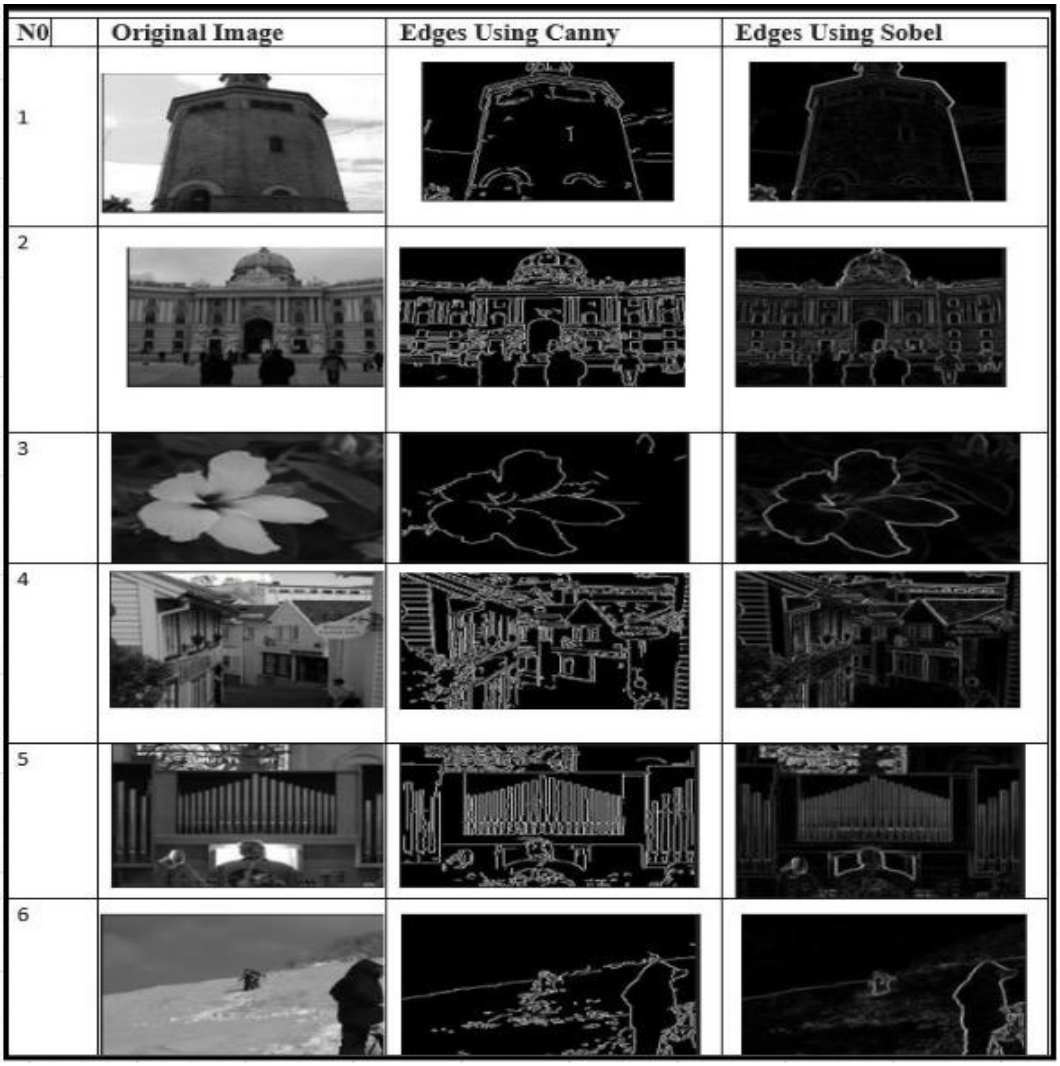



**FIGURE 1.** Edge detected areas using Canny and Sobel.

making it suitable for evaluating edge-based embedding strategies. This diversity ensures that the proposed method is robust and adaptable to various image types and conditions.

### B. EDGES DETECTION PROCESS USING CANNY AND SOBEL

In this stage, we choose 100 images for each dataset, and then we apply the canny and sobel algorithms separately to detect the edge regions on the cover images [32]. The sobel algorithm is considered as one of the mostly used filters for edge detection [33]. It works by calculating the gradient of the image intensity at each pixel, assuming the gradient measure of the rate of change of intensity in the image is high at edges. It also uses two 3 × 3 convolution kernels, i.e., horizontal changes ($G_X$) or x-direction kernel, and vertical changes ($G_Y$) or y-direction kernel, to measure the changes in both directions. So, the mathematical representation of sobel edge detector can be summarized as follows [34].

Let $I(i, j)$ be the grayscale intensity of the pixel at position $(i, j)$. As we mentioned before, the sobel operator uses two $3 \times 3$ kernels to estimate the gradient of the image, so the sobel kernels in the horizontal and vertical directions are:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \qquad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Additionally, the gradient magnitude at each pixel is calculated as:

$$G(i, j) = \sqrt{G_x(i, j)^2 + G_y(i, j)^2}$$

Moreover, the gradient direction (optional for edge orientation) is given by:

$$\theta(i, j) = \arctan\left(\frac{G_y(i, j)}{G_x(i, j)}\right)$$

The Sobel operator demonstrates superior performance in detecting areas with high spatial frequency, which typically

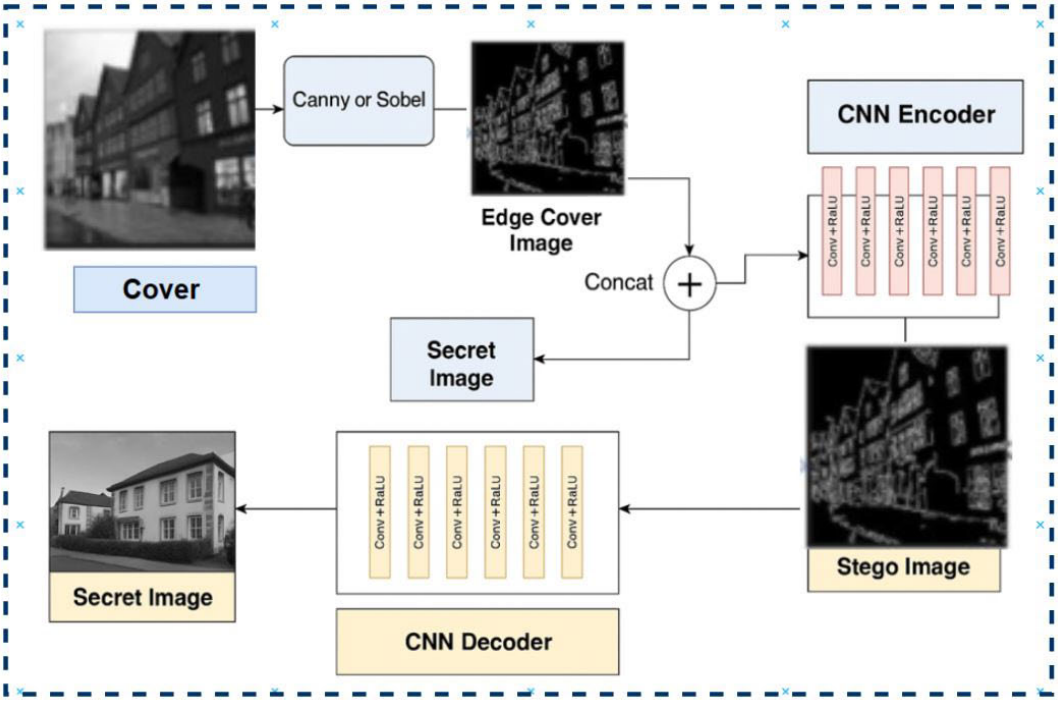



**FIGURE 2.** Block diagram of the proposed steganographic framework combining edge detection (Sobel/Canny) with a CNN-based embedding and extraction process.

indicate edges. The algorithm delivers efficient performance with optimal results that strike a balance between noise reduction and edge detection.

On other hand, canny edge detection algorithm is also one of the most efficient approaches for edge detection of images. This is because it provides precise edge detection of the original image with the least possible errors to give the best results in edge detection. The performance of edge detection when using canny is largely determined by the threshold used, where in our work we used a lower threshold of 100 and higher threshold of 200 [35]. In canny, the noise removal of the image is done by using a Gaussian filter. After that, we locate the edges by calculating the gradient of the image using sobel operators. Then, the non-maximum suppression is applied to reduce the edges to one-pixel width to achieve the desired outcome. Finally, canny classifies the pixels as either important or strong edge pixels, less important or weak edge pixels – based on gradient values. Moreover, the final edge detection where the weak edges that are not linked with the strong edges are removed. In noise reduction, we apply a Gaussian filter to smooth the image and reduce noise, so we use the following equation [36]:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Then, in gradient computation, we use sobel operators to compute the intensity gradients in both directions:

$$G = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan\left(\frac{G_y}{G_x}\right)$$

The algorithm refines detected edges by selecting the strongest edge points which follow the gradient direction to produce thinner and more precise edges. The algorithm uses two threshold values to determine edge pixel classification where strong edges remain unchanged, but weak edges need strong edge connections to be preserved. The algorithm completes the process by uniting weak and strong edges to create the final clean edge map while eliminating all isolated or noisy pixels. Figure 1 shows different images examples after applying sobel and canny algorithms to detect the edge regions.

### C. EMBEDDING AND EXTRACTION PROCESSES USING CNN

After generating the edge-detected cover images using canny and sobel, we combine them with secret images using the hiding network based on CNN. The edge-detected cover image and the secret image are used as input to CNN and concatenate them by passing them through convolutional layers that finally produce a stego image. Figure 2 shows the network architecture that is designed for concealing a secret image in edge-detected cover images. The cover image and secret image tensors are concatenated (stacked) along the channel dimension to create a single tensor that contains information from both images. This concatenation allows the hiding network to process both the cover and secret information together for embedding. The concatenated tensor is then passed through the hidden network, which performs the embedding process to generate the stego image.

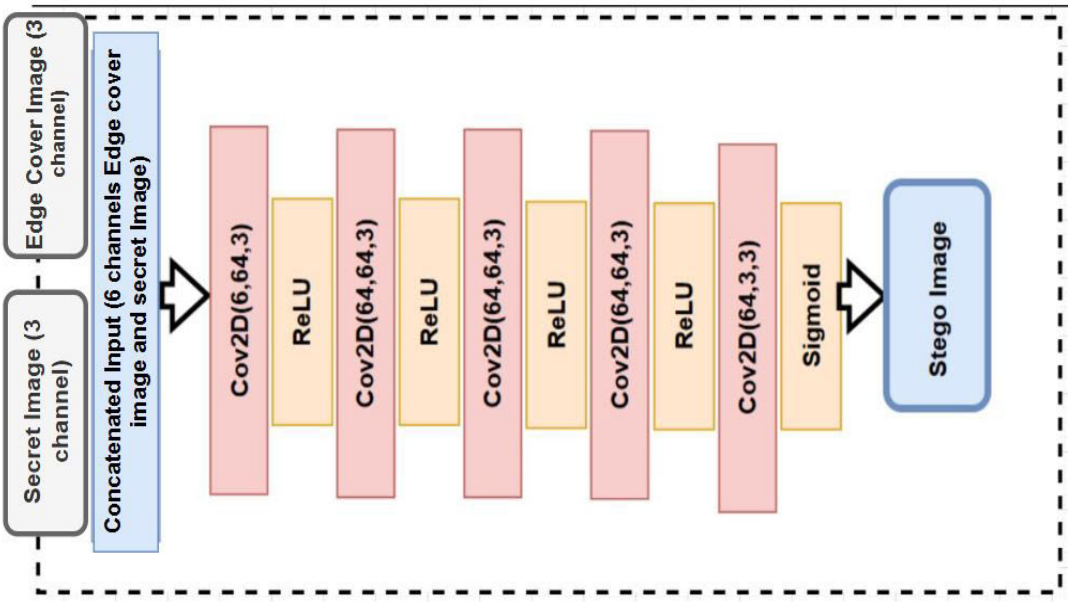



**FIGURE 3.** Architecture of the proposed CNN-based encoder for steganography. The input consists of two 3-channel images (cover and secret), concatenated into a 6-channel input.

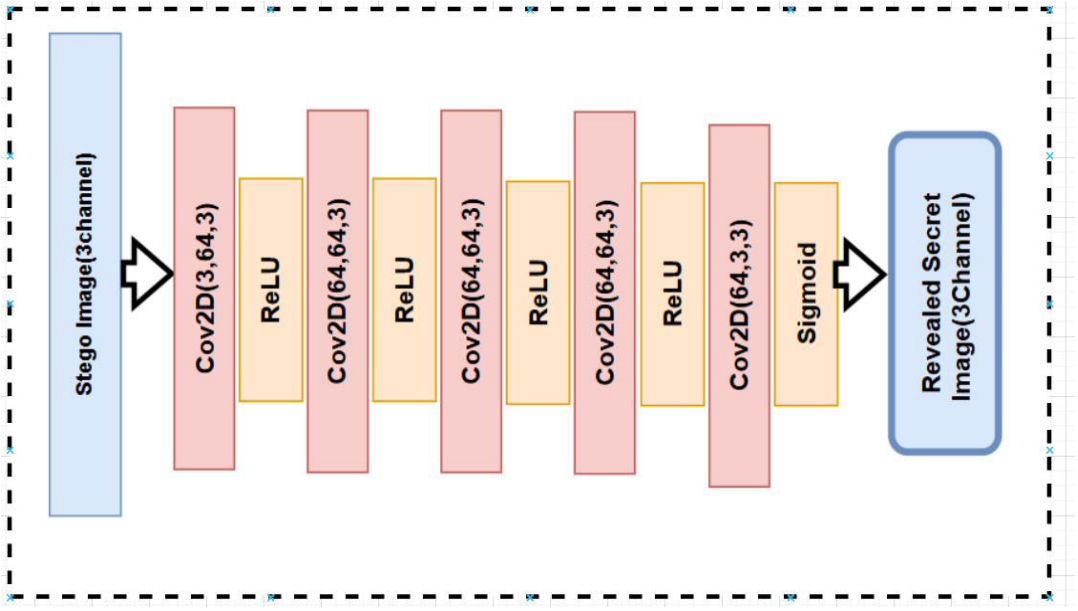



**FIGURE 4.** Revealing network architecture based on CNN for secret image extraction. The input is the 3-channel stego image, which is passed through a series of Conv2D layers with ReLU activations.

Through concatenating the tensors, the hiding network can access information from both the cover and secret images simultaneously during the embedding process. The hiding network leverages the concatenated tensor, which includes edge information from the cover image and pixel data from the secret image, to embed the secret information within the edges.

Figure 3 shows the hiding network that is composed of several convolutional layers and ReLU activation functions. The input to the hiding network is 6 channels, of which 3 channels are for the cover image and 3 channels are for the secret image. The last convolutional layer downsamples the output channels to 3, which can be used as the output image (i.e. the stego image) using the sigmoid activation function. On the other hand, the revealing network uses a CNN that takes the stego image (from the Hiding Network) through convolutional layers to extract the secret image from the stego image. Figure 4 shows the revealing network architecture that

is planned to decode the hidden information and generate the original secret image. The revealing network is constructed as a sequence of convolutional layers, each followed by a ReLU activation. The input to the revealing network has 3 channels, which might serve as the stego image, learned by the hiding network. The revealing network architecture is built upon several layers of 2D convolution, followed by ReLU activation. The last convolutional layer has output channels of 3, which can be interpreted as the output image (i.e., the revealed secret image). The output of the revealing network is passed through a sigmoid activation function to make sure that the pixel values are in the range [0,1]. The training process iterates through multiple epochs, where each epoch consists of iterating through the dataset in batches. For each batch, the model calculates the loss, performs back-propagation to compute gradients, and updates the model parameters using the optimizer. The training process is done based on both the hidden network and the revealing network. During the training loop, the hiding network is used to hide the secret image within the cover image to create the stego image. The loss is calculated based on how well the stego image resembles the cover image. The loss is computed using Mean Square Error (MSE) between the stego and cover image. The Adam optimizer updates the parameters of the hiding network based on this loss, ensuring that the hiding network learns to embed the image effectively. On the other hand, the revealing network is responsible for extracting the secret image from the stego image. During the training loop, the revealing network is used to reveal the secret image from the stego image. The loss is calculated using the MSE between the revealed image and the original secret image. The Adam optimizer updates the parameters of the revealing network based on this loss, ensuring that the revealing network learns to extract the secret image accurately.

**TABLE 2. Parameters of training with values.**

| Parameters of Training | Values |
|---|---|
| Batch Size | 8 |
| EPOCH | 300 |
| Learning Rate | 0.0001 |
| Loss Function | MSE |
| Optimizer | Adam |

Table 2 shows the parameters that are used in the training process. In training a CNN, the epoch is the number of times the training data has been executed through the network. The batch size is the number of samples processed in each iteration of training, which impacts the training speed and stability. The learning rate is the magnitude by which the model weights are adjusted during the training process, which affects the convergence rate and accuracy. The optimizer is the method of updating model parameters, which is important for efficient learning. The loss function is the model performance function that computes the error between the prediction and the actual label to guide the network to better results. These parameters must be balanced optimally for effective training of a CNN for better performance and generalization.

The CNN architecture used in this study was designed to balance performance and computational efficiency. While relatively simple in design (consisting of sequential convolutional layers with ReLU activations), it achieved high PSNR and SSIM scores across datasets. Preliminary tests with deeper models (e.g., additional convolutional blocks or residual connections) showed marginal performance gains but increased training complexity and runtime. Thus, we selected a compact architecture that is well-suited for real-time or resource-constrained scenarios. Future work may explore more advanced architectures such as U-Net, and the selection of a moderately deep CNN architecture based on U-Net may ensure more practical performance stability and computational efficiency. The U-Net architecture with attention mechanisms and dual input streams provides sufficient capability to extract both local and global image features for our objective of accurate and adaptive embedding map generation. Though the use of ResNet or transformer-based architectures for deeper models would provide better feature extraction capabilities yet they increase computational complexity and need larger datasets and longer training times for effective convergence. However, the chosen design in this article enables us to achieve high steganographic security while maintaining efficiency and stability during training.

## IV. EXPERIMENT AND RESULTS

There are different measures i.e., imperceptibility, capacity, security, and robustness, that show the performance of the proposed strategy. To measure the imperceptibility, we use different metrics such as mean square error (MSE), root mean square error (RMSE), peak signals to noise ratio (PSNR), structural similarity index (SSIM), and image fidelity (IF). To measure the capacity, we use payload capacity (PC) and bits per pixel (BPP), and to measure robustness, we use error rate extraction (Re) and PSNR. To measure the security, we use two blind deep learning algorithms (Xu-Net and Ye-Net) that work as steganography methods. The details of these performance measurement metrics are described below.

### A. IMPERCEPTIBILITY

#### 1) MSE

It is used to represent the average of the mean square error between the pixels of the cover image and the stego image where the MSE value is calculated by using Equation 1. The range of MSE value is between 0 and 1, and the lower values (that are close to zero) are better than the higher values (that are closer to one) [37].

$$\text{MSE} = \frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}(I(i,j) - K(i,j))^2 \tag{1}$$

where:

MSE : Mean Squared Error
$m, n$ : Image dimensions
$I(i, j)$ : Pixel value of the original image
$K(i, j)$ : Pixel value of the stego image

#### 2) RMSE

As a result of the quadratic process, the square root is applied in MSE to obtain RMSE. Hence, a smaller value will be acquired, but this value is correlated with the MSE value. The RMSE can be computed from the following Equation 2:

$$\text{RMSE} = \sqrt{\text{MSE}} \tag{2}$$

#### 3) PSNR

It is used to describe the quality of the stego-image based on the HSV with its normal value as 30 dB [38]. When the PSNR value is greater than 30, it means that the data inserted in the image is invisible to the human eye [10]. The PSNR value can be calculated by Equation 3, and the higher values of PSNR are better than the lower values.

$$\text{PSNR} = 10 \cdot \log_{10}\left(\frac{\text{MAX}^2}{\text{MSE}}\right) \tag{3}$$

where:

PSNR : Peak Signal-to-Noise Ratio
MAX : Maximum possible pixel value
MSE : Mean Squared Error between original and stego image

#### 4) SSIM

It is used to measure the similarity between the cover image and the stego image. The result of the SSIM value remains between 0 and 1. When the SSIM value is close to 1, then it means that the stego image is similar to the cover image and is of high quality. The SSIM value is calculated using Equation 4.

$$\text{SSIM}(I, K) = \frac{(2\mu_I\mu_K + C_1)(2\sigma_{IK} + C_2)}{(\mu_I^2 + \mu_K^2 + C_1)(\sigma_I^2 + \sigma_K^2 + C_2)} \tag{4}$$

where:

$I, K$ : Two different images
$\text{SSIM}(I, K)$ : Structural Similarity Index Measure
$\mu_I, \mu_K$ : Mean intensity of images $I$ and $K$
$\sigma_I, \sigma_K$ : Standard deviation of intensities of images $I$ and $K$
$\sigma_{IK}$ : Covariance of intensities of images $I$ and $K$
$C_1, C_2$ : Constants to stabilize the division

#### 5) IF

It is another metric to identify the quantity of errors in pixel values. If the value is closer to 1, it means that the two images are more similar [39]. The method of calculation is shown in Equation 4 as follows.

$$\text{IF}(I, K) = \frac{1}{N}\sum_{i=1}^{N}\left(1 - \frac{|I_i - K_i|}{\max(I_i, K_i)}\right) \tag{5}$$

where:

$\text{IF}(I, K)$ : Information Fidelity between images $I$ and $K$
$N$ : Total number of pixels in the images
$I_i, K_i$ : Pixel values at position $i$ in images $I$ and $K$

**TABLE 3. Performance of imperceptibility metrics for different datasets with Canny and Sobel operators.**

| Dataset | MSE | RMSE | PSNR | SSIM | IF |
|---|---|---|---|---|---|
| Bossbase + Canny | 0.00074 | 0.02725 | 31.29 dB | 0.988 | 0.9686 |
| Tiny_ML + Canny | 0.0004 | 0.0221 | 33.08 dB | 0.995 | 0.994 |
| USC_SIPI + Canny | 0.00090 | 0.03004 | 30.4 dB | 0.99 | 0.98 |
| Bossbase + Sobel | 0.00010 | 0.0101 | 39.85 dB | 0.997 | 0.989 |
| Tiny_ML + Sobel | 0.00077 | 0.0278 | 31.08 dB | 0.996 | 0.993 |
| USC_SIPI + Sobel | 0.0013 | 0.0361 | 28.84 dB | 0.981 | 0.969 |

Table 3 shows that the Bossbase dataset with canny edge detection has low MSE and RMSE values, good PSNR, and high SSIM and IF values, which indicate that the image quality and similarity between the original cover and stego images are high. In the case of Sobel edge detection, the results are even better than that in Canny, with significantly lower MSE, RMSE, higher PSNR, SSIM and IF values. The Tiny ImageNet dataset in conjunction with both Canny and Sobel edge detection methods is quite efficient, with respect to all the metrics used, having very low MSE and RMSE values, good PSNR, high SSIM, and IF values. Both edge detection methods appear to give good results with this dataset. The USC_SIPI dataset with Canny edge detection is also quite efficient with low MSE and RMSE values, good PSNR, high SSIM and IF values. But using the Sobel edge detection method with this dataset is a little worse than Canny, with higher MSE and RMSE values and lower PSNR, SSIM and IF values. The results also show that images with lower MSE values are more likely to have better image quality and less distortion. The higher the PSNR value the better the image quality. The SSIM values are also found to be highest, which shows that the images are similar. IF is the measure of the quantity of information conserved during processing, which implies that a large portion of information is conserved. In these metrics, we note that because the sobel operator is a simple gradient-based edge detection while compared with canny and uses multiple stages to detect edges, it can lead to better PSNR values since edges are detected more precisely. Also, the use of thresholding to remove weak edges and retain strong edges can aid in

avoiding false positives and negatives and can be useful in canny edge detection. This thresholding process of canny can improve the PSNR values as compared with sobel (which does not include such sophisticated thresholding techniques).

### B. CAPACITY

#### 1) PC

It is a measure of how much data can be hidden in the cover image by the proposed steganography, and it will be quite dependent on the size of cover image [40]. Equation 6 shows the calculation of this measure.

$$\text{Payload Capacity} = \text{T} \times N \times 8 \tag{6}$$

where:

T : Total pixels of cover image

$N$ : bytes per pixels: 1 for grayscale or 3 for colour image

#### 2) BPP

It is the average value of the embedding per pixel of the whole image [41]. BPP can be calculated by Equation 7.

$$\text{BPP} = \frac{\text{Payload Capacity (bits)}}{N} \tag{7}$$

where:

BPP : Bits Per Pixel

Payload Capacity (bits) : Total payload capacity in bits

$N$ : Total number of pixels in the image

**TABLE 4. Performance of capacity metrics for different datasets with Canny and Sobel operators.**

| Dataset | PC | BPP |
|---|---|---|
| Bossbase + Canny | 1572864 bits | 8 |
| Tiny_ML + Canny | 1572864 bits | 8 |
| USC_SIPI + Canny | 1572864 bits | 8 |
| Bossbase + Sobel | 1572864 bits | 8 |
| Tiny_ML + Sobel | 1572864 bits | 8 |
| USC_SIPI + Sobel | 1572864 bits | 8 |

Table 4 shows the results that are consistent in the number of bits (1572864 bits) where the number of bits per pixel is equal to 8. This means that the Canny and sobel edge detection techniques have the same data processing efficiency (in bits). The homogeneous values of the variables suggest that the selection of edge detection method (canny or sobel) does not affect the processing requirements essentially.

### C. ROBUSTNESS

#### 1) RE

This measuring tool is used to determine the bit error that is in the extracted message and it is used to measure the robustness of the methods against attacks. When the error value is close to 0, it means that the error is becoming smaller; when it is zero, then it means that the extraction is perfect [42]. It can be calculated by Equation 8.

$$\begin{aligned} e &= \frac{N_e}{LM} \\ R_e &: \text{Error Rate} \\ N_e &: \text{Total number of errors} \\ LM &: \text{Length of the message} \end{aligned} \tag{8}$$

where $N_e$ is the number of error message bits, $LM$ is the total message bits. The robustness of our proposed strategy is discussed next.

### D. ROBUSTNESS OF OUR APPROACH AGAINST ATTACKS

The media channels could suffer several attacks. Steganography model robustness is defined by the ability of the model to extract the hidden data from the attacked stego with no significant damage. There are different types of attacks, such as a *JPEG compression attack* that is used to decrease the size of an image file with a quality factor ranging between 10 to 90. The second attack is a *Gaussian noise attack* that is obtained by adding random noise from the Gaussian distribution with a variance value $\sigma$ between 0.001 and 0.005. The third attack is *salt and pepper noise* that randomly picks some pixels and sets them to either 0 or 255 with a variance of 0.001. The fourth attack is *speckle noise* that randomly disperses white and black pixels with a variance value between 0.01 and 0.1. The fifth attack is *median filtering* that replaces the value of a pixel using the median value of the surrounding windows with a size of $3 \times 3$. To measure the robustness of our method against these attacks, we consider the measures, such as Re and PSNR. Again, the performance was measured for various compression levels.

#### 1) COMPRESSION ATTACK

The compression was performed using different quality factors denoted by Q, which represent the level of image quality after compression. A higher value such as Q(90) indicates light compression with minimal visual degradation, while a lower value such as Q(70) corresponds to moderate compression with increased loss of detail. Evaluating the system under varying Q values provides an effective measure of its resilience against distortions that may occur during image storage or transmission in non-ideal environments.

Table 5 shows that for canny and sobel edge detection methods, the Re value (error extraction rate) is 0.47 and 0.34 respectively for both edge detectors in the Bossbase dataset for different quality factors. For both canny and sobel edge detection methods, the Tiny ImageNet dataset has a stable Re value of 0.4171 and 0.34, respectively, for various compression levels. The USC_SIPI dataset also maintains a constant Re value of 0.32 for canny and 0.34 for sobel

**TABLE 5.** Performance of Re robustness metric for different datasets with Canny and Sobel operators against compression attack.

| Dataset | Q(10) | Q(20) | Q(30) | Q(40) | Q(50) | Q(60) | Q(70) | Q(80) | Q(90) |
|---|---|---|---|---|---|---|---|---|---|
| Bossbase + Canny | 0.47 | 0.47 | 0.47 | 0.47 | 0.47 | 0.47 | 0.47 | 0.47 | 0.47 |
| Tiny_ML + Canny | 0.4171 | 0.4171 | 0.4171 | 0.4171 | 0.4171 | 0.4171 | 0.4171 | 0.4171 | 0.4171 |
| USC_SIPI + Canny | 0.32 | 0.32 | 0.32 | 0.32 | 0.32 | 0.32 | 0.32 | 0.32 | 0.32 |
| Bossbase + Sobel | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 |
| Tiny_ML + Sobel | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 |
| USC_SIPI + Sobel | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 | 0.34 |

**TABLE 6.** Performance of PSNR robustness metric for different datasets with Canny and Sobel operators against compression attack.

| Dataset | Q(10) | Q(20) | Q(30) | Q(40) | Q(50) | Q(60) | Q(70) | Q(80) | Q(90) |
|---|---|---|---|---|---|---|---|---|---|
| Bossbase + Canny | 67.3 dB | 67.3 dB | 67.3 dB | 67.3 dB | 67.3 dB | 67.3 dB | 67.3 dB | 67.3 dB | 67.3 dB |
| Tiny_ML + Canny | 66.8 dB | 66.8 dB | 66.83 dB | 66.8 dB | 66.8 dB | 66.8 dB | 66.8 dB | 66.8 dB | 66.8 dB |
| USC_SIPI + Canny | 76.2 dB | 76.2 dB | 76.2 dB | 76.2 dB | 76.2 dB | 76.2 dB | 76.2 dB | 76.2 dB | 76.2 dB |
| Bossbase + Sobel | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB |
| Tiny_ML + Sobel | 70.5 dB | 70.5 dB | 70.5 dB | 70.5 dB | 70.5 dB | 70.5 dB | 70.5 dB | 70.5 dB | 70.5 dB |
| USC_SIPI + Sobel | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB | 73.6 dB |

**TABLE 7.** Performance of robustness metrics for different datasets with Canny and Sobel operators against *Gaussian noise* with factors (0.001 and 0.005).

| Dataset | Re ($\sigma^2$=0.001) | PSNR ($\sigma^2$=0.001) | Re ($\sigma^2$=0.005) | PSNR ($\sigma^2$=0.005) |
|---|---|---|---|---|
| Bossbase + Canny | 0.5522 | 54.0253 dB | 0.5523 | 54.0231 dB |
| Tiny_ML + Canny | 0.4775 | 55.8051 dB | 0.4770 | 55.7916 dB |
| USC_SIPI + Canny | 0.4661 | 55.8562 dB | 0.4663 | 56.0695 dB |
| Bossbase + Sobel | 0.5179 | 54.3804 dB | 0.5180 | 54.3759 dB |
| Tiny_ML + Sobel | 0.5043 | 57.0947 dB | 0.5031 | 57.1217 dB |
| USC_SIPI + Sobel | 0.4856 | 54.8602 dB | 0.4856 | 54.8607 dB |

edge detection methods for different levels of compression. The USC_SIPI dataset has the lowest Re values in all the edge detection methods, thus being more robust to compression attacks. The Bossbase dataset, in general, has higher Re values than the other datasets, which suggests that it is less robust to compression attacks. The Re values are constant with varying compression levels for each dataset and each edge detection method, which indicates that the robustness of steganography against compression attacks is constant. So, USC_SIPI is better than the Bossbase and Tiny imageNet dataset in terms of the robustness to compression attacks.

In Table 6, under canny edge detection method, the Bossbase dataset has a consistent PSNR value of 67.3 dB for all the compression levels. With the sobel edge detection method, the PSNR is also constant at 73.6 dB for all the compression levels. The Tiny ImageNet dataset with canny edge detection method has PSNR values of approximately 66.8 dB for different compression levels. In the sobel edge detection method, the PSNR remains at 70.5 dB for all the compression levels. The USC_SIPI dataset has a very high PSNR of 76.2 dB for canny and 73.6 dB for sobel edge detection methods, and maintains this consistency over different levels of compression. The Bossbase dataset is generally smaller than other datasets and has relatively low PSNR values, which indicates that it is less robust to compression attacks. There is a significant discrepancy in

**TABLE 8.** Performance of robustness metrics for different datasets with Canny and Sobel operators against Salt_and_Pepper_noise attack with factor (0.001).

| Dataset | Re ($\sigma^2$=0.001) | PSNR ($\sigma^2$=0.001) |
|---|---|---|
| Bossbase + Canny | 0.5467 | 54.2494 dB |
| Tiny_ML + Canny | 0.4781 | 56.0371 dB |
| USC_SIPI + Canny | 0.4667 | 56.0668 dB |
| Bossbase + Sobel | 0.5162 | 54.6266 dB |
| Tiny_ML + Sobel | 0.5083 | 57.1383 dB |
| USC_SIPI + Sobel | 0.4850 | 54.8407 dB |

**TABLE 9.** Performance of robustness metrics for different datasets with Canny and Sobel operators against speckle_noise attack.

| Dataset | Re ($\sigma^2$=0.01) | PSNR ($\sigma^2$=0.01) | Re ($\sigma^2$=0.1) | PSNR ($\sigma^2$=0.1) |
|---|---|---|---|---|
| Bossbase + Canny | 0.5525 | 54.0232 dB | 0.5519 | 54.0174 dB |
| Tiny_ML + Canny | 0.4753 | 55.7568 dB | 0.4752 | 55.8247 dB |
| USC_SIPI + Canny | 0.4665 | 56.0857 dB | 0.4658 | 56.0518 dB |
| Bossbase + Sobel | 0.5176 | 54.3848 dB | 0.5184 | 54.3808 dB |
| Tiny_ML + Sobel | 0.5075 | 57.1128 dB | 0.5028 | 57.2220 dB |
| USC_SIPI + Sobel | 0.4848 | 54.8541 dB | 0.4860 | 54.8411 dB |

**TABLE 10.** Performance of robustness metrics for different datasets with Canny and Sobel operators against median filter attack.

| Dataset | BER | Re | PSNR |
|---|---|---|---|
| Bossbase + Canny | 54.6939% | 0.5469 | 54.2427 dB |
| Tiny_ML + Canny | 47.4731% | 0.4747 | 55.9263 dB |
| USC_SIPI + Canny | 46.5640% | 0.4656 | 55.8182 dB |
| Bossbase + Sobel | 51.6800% | 0.5168 | 54.6317 dB |
| Tiny_ML + Sobel | 50.3784% | 0.5038 | 56.9270 dB |
| USC_SIPI + Sobel | 48.4281% | 0.4843 | 54.9344 dB |

**TABLE 11.** Performance of Stegnalysis metrics for different datasets with Canny and Sobel operators.

| Dataset | Xu_Net | Yu_Net |
|---|---|---|
| Bossbase + Sobel | 0.62 | 0.47 |
| Tiny_ML + Sobel | 0.52 | 0.52 |
| USC_SIPI + Sobel | 0.67 | 0.52 |
| Bossbase + Canny | 0.52 | 0.52 |
| Tiny_ML + Canny | 0.52 | 0.52 |
| USC_SIPI + Canny | 0.55 | 0.45 |

PSNR values between the canny and sobel edge detection methods for all the datasets. From the comparison of the different datasets, it is evident that USC_SIPI performs better than Bossbase and Tiny ImageNet datasets in terms of robustness to compression attacks as measured by PSNR.

#### 2) GUASSIAN NOISE ATTACK

Table 7 shows the results of a Gaussian noise attack with a factor or variance value, $\sigma^2$ (the variance factor indicates with how much variation the noise was added to the image) equal to 0.001 (where the Re is 0.5522 and the PSNR is 54.0253 dB) and for Gaussian noise with a factor 0.005 (where the Re is 0.5523 and the PSNR is 54.0231 dB). The Re and PSNR values of the Bossbase dataset are similar for both noise levels. The PSNR values are also relatively higher for the Tiny ImageNet Dataset than the Bossbase. In all the metrics, Tiny ImageNet Dataset performs better than Bossbase. When the noise level is increased from 0.001 to 0.005, there is a slight decrease in the performance of all the datasets and edge detection methods as evidenced by the decrease in PSNR. The USC_SIPI dataset with canny is the least affected by Gaussian noise attacks and has the highest PSNR values, and the Tiny ImageNet Dataset is the least affected when sobel was used.

#### 3) SALT AND PEPPER NOISE ATTACK

Table 8 displays the salt and pepper noise attack while using canny and sobel edge detection methods (the Re is 0.5467 and the PSNR is 54.2494 dB with a noise factor of 0.001). In the Tiny ImageNet dataset, canny edge detection method shows Re values of around 0.4781 and PSNR values of around 56.0371 dB, and sobel edge detection method shows Re values of around 0.5083 and a higher PSNR value as 57.1383 dB. The USC_SIPI dataset while using canny is also also more robust than others against the salt and pepper noise attack based on the reasonable PSNR values.

#### 4) SPEKEL NOISE ATTACK

For a speckle noise attack with factor 0.1, the values are Re 0.5519 and PSNR 54.017 dB. In Table 9, a speckle noise attack with factor 0.01 on the Bossbase dataset has Re of 0.5525 and PSNR of 54.0232 dB. In the case of the sobel edge detection method, the Bossbase dataset exhibits similar trends in performance for both noise levels. The Tiny ImageNet dataset has the lowest Re value about 0.4753 for canny, and higher PSNR values of about 55.7568 dB for canny and 57.1128 dB for sobel edge detection method. USC_SIPI also has the higher PSNR value with sobel, but the Tiny ImageNet dataset with sobel has the highest resistance against speckle noise levels.

#### 5) MEDIAN FILTER ATTACK

Table 10 shows that the Tiny ImageNet dataset is better suited for canny edge detection as it shows a higher PSNR value of 55.9263 dB in comparison to sobel (56.9270 dB). Canny (55.8182 dB) outperforms sobel (54.9344 dB) on the USC_SIPI dataset, indicating that canny is more suitable for this dataset. Canny produces a PSNR of 54.2427 dB and sobel produces a PSNR of 54.6317 dB for the Bossbase dataset thus, sobel has a slight advantage here. The USC_SIPI dataset shows nearly similar PSNR values in both edge detection techniques, which indicates that the dataset is quite insensitive to edge detection methods. The datasets are also observed with Re value and BER, where variations in PSNR values can be mainly noted between the edge detection methods.

### *E. SECURITY AGAINST STEGNALYZER*

Steganalysis is the process of breaking steganography by detecting the existence of a secret message hidden in the cover image. Since it is a classification problem, deep learning models give accurate detection results. One of the most famous CNN-based steganalyses is Xu Net proposed by Xu et al. [51]. The structure of Xu Net is depicted

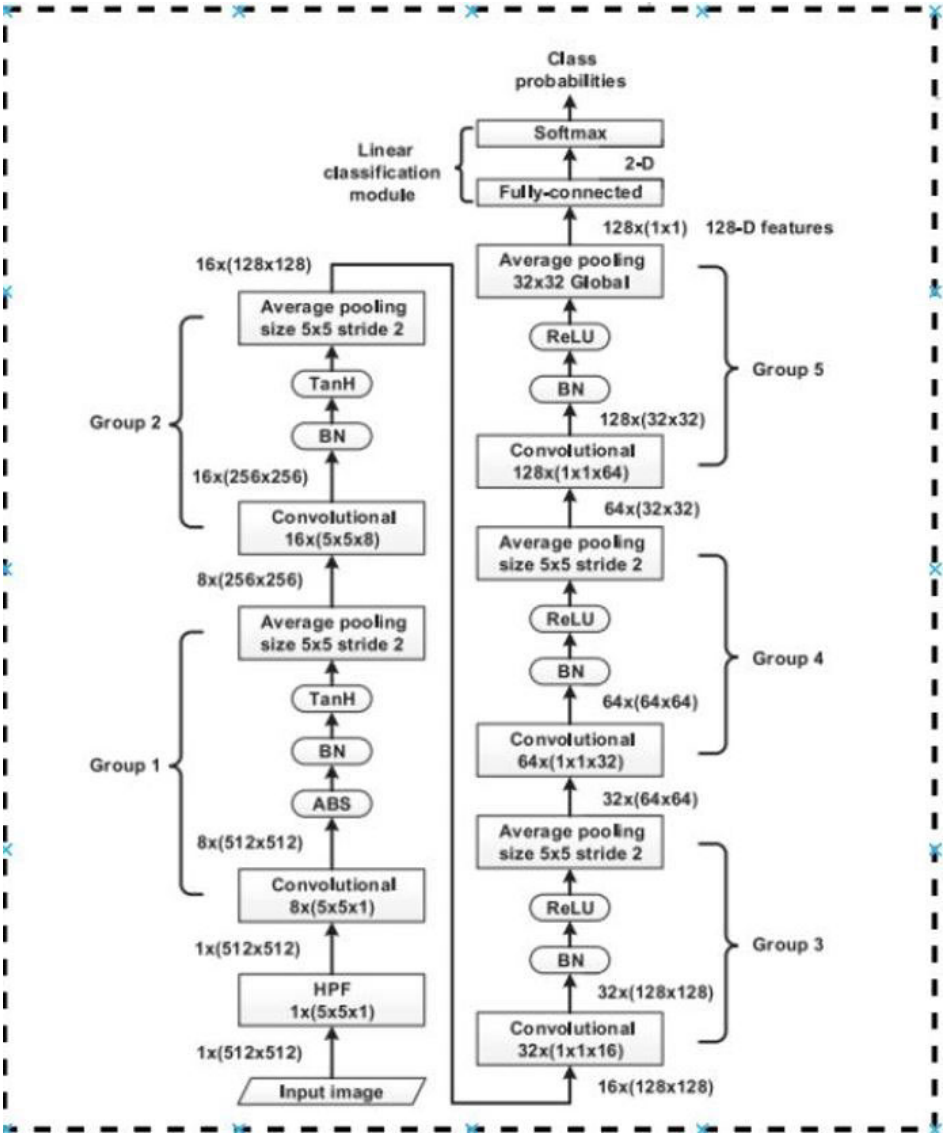



**FIGURE 5.** Xu Net architecture (G. Xu et al.) [45].

in Figure 5. It has several 5 × 5 convolutional layers, batch normalization, and average pooling. First, they also incorporated the absolute activation function after the first convolutional layer to enhance the statistical modelling. In addition, they employed 1 × 1 convolutional layers in deeper groups to prevent overfitting. Another typical CNN-based steganalysis is Ye Net suggested by Ye et al. [52] as shown in Figure 6. Compared with other methods, Ye Net has more convolutional layers. They used nine convolutional blocks and reduce the number of parameters by using one fully connected layer. In addition, they did not employ a random weight initialization but instead employed a high-pass filter to initialize the weight of the first layer.

Table 11 compares the performance of the Xu_Net and Yu_Net models on a variety of datasets using sobel and canny edge detection methods. When the Bossbase dataset is processed using the sobel edge detection method, the Xu_Net model performs better than the Yu_Net model. For instance, on the Xu_Net model, the accuracy is 0.62, while that of Yu_Net is 0.47. When the Tiny ImageNet dataset was analyzed using sobel, the Xu_Net and Yu_Net models had equal accuracy of 0.52. When the datasets were processed using canny edge detection, the performance was quite mixed. Both the Xu_Net and Yu_Net models have the same accuracy of 0.52 in the Bossbase and Tiny_ML datasets, but in the USC_SIPI dataset, the Xu_Net model performs better than the Yu_Net model, with the scores of 0.55 and 0.45, respectively. The results of the experiments show that the Xu_Net model is better than the Yu_Net model in most cases, especially when the sobel edge detection method is used. The similarity in the performance of Xu_Net and

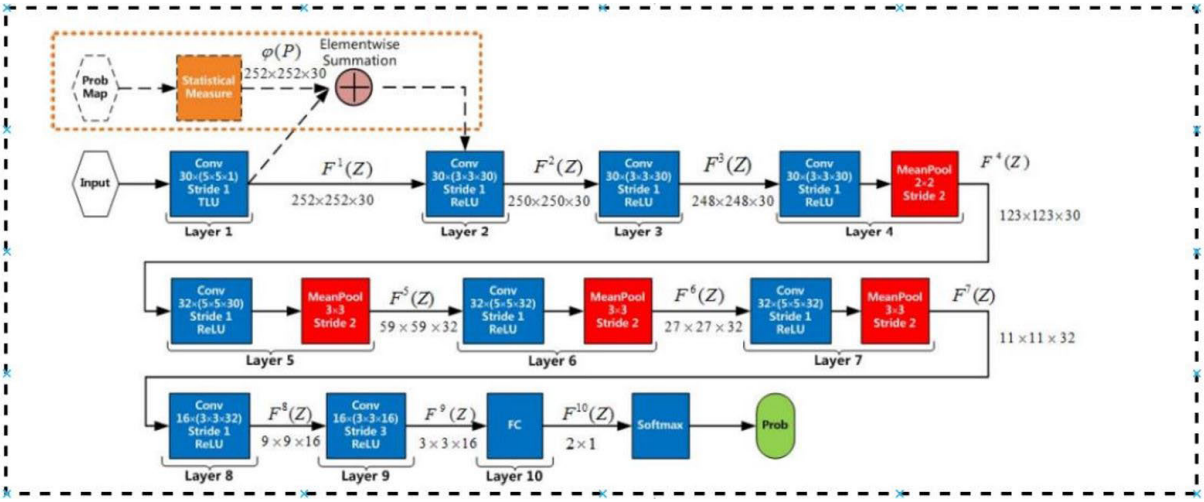



**FIGURE 6.** Ye Net architecture (Ye et al.) [46].

**TABLE 12.** Comparison between our work and related works based on USC-SIPI dataset.

| Paper | Dataset | Imperceptibility | Capacity | Security | Robustness |
|---|---|---|---|---|---|
| [43] | USC-SIPI | PSNR=4 dB6 dB SSIM=0.99 | Tested Up to 4BPP | Stego Expose=Secure, Sample Pair=Not secure | N/A |
| [42] | USC-SIPI | PSNR=3 dB9 dB SSIM=0.98 | BPP=3.1 PC=800000bits | RS=secure, PDH=Secure, SPAM=Secure | BER=46%, Salt and pepper Attack |
| [44] | USC-SIPI | PSNR=6 dB9 dB SSIM=0.99 | PC=49152 Bytes | Chi-square=secure | N/A |
| [45] | USC-SIPI | PSNR=29 dB to 36 dB SSIM=0.99 | BPP=1.9 to 3.2 | Histogram=Secure | N/A |
| Proposed Work | USC-SIPI | PSNR=28 dB to 30 dB SSIM=0.99 MSE=0.0009 IF=0.98 | PC=572864 bits BPP=8 | XU-Net=Secure YU-Net=Secure, | Robust under all attacks |

Yu_Net on the Tiny ImageNet dataset using sobel shows that both models are equally efficient in this particular context. The different performance on datasets filtered with canny edge detection may show that the selection of edge detection method can affect the comparison between the Xu_Net and Yu_Net models. In this paper, we note that canny gives better results because it has higher-quality edge maps. With canny, the model conceals the presence of hidden information within the edge areas by making it more challenging for steganalyzer models such as Xu-Net and Ye-Net to detect it.

### F. COMPARISON WITH PREVIOUS WORK

Table 12 shows that the proposed framework has competitive imperceptibility, higher capacity, enhanced security features, and robustness in all attack scenarios, thus making it a promising solution for data-hiding applications. There is no doubt that some of the existing methods are good in some particular aspects, for instance, imperceptibility or capacity, but they do not have the all-around security and robustness. Therefore, the proposed work presents a well-rounded approach with a balance between imperceptibility, capacity, security, and robustness that makes it a strong contender in the field of data-hiding techniques.

Table 12 shows that the work presented in the Bossbase dataset achieves strong imperceptibility, a large capacity, additional security features, and resistance to all types of attacks, thus providing a complete and reliable solution for data-hiding applications. The methods presented in the dataset have advantages in terms of imperceptibility, capacity, and security, but they do not have all the robustness of the proposed work. Based on the analysis, the proposed work in the Bossbase dataset has been found to have a holistic approach to the problem, providing high imperceptibility, high capacity, security features, and rrobustness, making it a viable and effective method among other data hiding techniques.

**TABLE 13. Comparison between our work and related works based on USC-SIPI dataset.**

| Paper | Dataset | Imperceptibility | Capacity | Security | Robustness |
|---|---|---|---|---|---|
| [46] | Bossbase | PSNR>35 | 0.05-0.5 bpnzAC | DCTR=secure, GFR=secure, | Error rate from JPEG compression Q 75,Q95 |
| [47] | Bossbase | PSNR>50 | 0.01-0.1 bpnzAC | DCTR=secure, GFR=secure, | Error rate from JPEG compression Q 65,Q75,Q 85 |
| [48] | Bossbase | PSNR>52 | 0.05-0.3 bpnzAC | DCTR=secure, GFR=secure, | Error rate from JPEG compression Q 75,Q85,Q 90 |
| [49] | Bossbase | N/A | 0.1-0.5 bpnzAC | DCTR=secure, SR net=secure, | Error rate from JPEG compression Q 70 |
| [50] | Bossbase | Not reported | Not Reported | Achieved the lowest detection accuracy across five steganalyzers (SRM, Deng-Net, SRNet, Yedroudj-Net, YeNet), improving security by up to 5.39% over prior methods | Not Reported |
| Proposed Work | Bossbase | MSE=0.0007 to 0.001, PSNR=33 dB to 39, SSIM=0.99, IF=0.99 | PC=572864 bits, BPP=8 | XU-Net=Secure, YU-Net=Secure, | Robust under all attacks |

## V. CONCLUSION

We have developed an image steganography system in this study where the cover image is first analyzed by Canny and Sobel edge detection algorithms to determine the regions where the hidden data is to be embedded or retrieved using Convolutional Neural Networks (CNN). The system places the data in the cover image's edges to guarantee that the hidden information is well protected and not easily detectable. The use of CNN in the process of embedding and extracting information into and from the image has improved the efficiency and strength of the steganographic system to enable clean and clear data encoding and decoding without affecting the quality of the image. The system achieved high imperceptibility through PSNR values of 39.85 dB and SSIM values of 0.997 while maintaining strong capacity (BPP $= 8$) and effective resistance against common image processing attacks. Security analysis against Xu-Net and Ye-Net revealed low detectability rates, which confirmed the stealthiness of the proposed method. But the proposed framework has several limitations due to high dependence on conventional edge detection algorithms and it tends to fail in producing optimal results when dealing with images that have ambiguous or faint edges. Our embedding approach also lacks flexibility to handle variations in image complexity, which negatively impacts its performance on particular image types. The CNN achieves good results, but its performance falls short of what newer deep learning models such as GANs and Transformers could deliver in terms of accuracy and generalization. Again, the evaluation process was restricted to three datasets, while the system remained untested in difficult real-world scenarios and further adversarial conditions.

## FUTURE WORK

The future work for a steganographic approach based on edge detection and Convolutional Neural Networks (CNNs) includes increasing the security and robustness of the approach through the integration of advanced encryption techniques; increasing the detection resistance against additional steganalysis algorithms; examining different embedding strategies inspired by edge features; try out different CNN architectures including GAN and Autoencoders for steganography. Finally,future work will explore adaptive payload allocation, integration with deeper and more advanced architectures, and broader testing against emerging steganalysis techniques in more diverse image domains.

## REFERENCES

[1] Z. Wang, G. Feng, Z. Qian, and X. Zhang, "JPEG steganography with content similarity evaluation," *IEEE Trans. Cybern.*, vol. 53, no. 8, pp. 5082–5093, Aug. 2023.

[2] B. Ray, S. Mukhopadhyay, S. Hossain, S. K. Ghosal, and R. Sarkar, "Image steganography using deep learning based edge detection," *Multimedia Tools Appl.*, vol. 80, no. 24, pp. 33475–33503, Oct. 2021.

[3] A. Das, J. Singh Wahi, M. Anand, and Y. Rana, "Multi-image steganography using deep neural networks," 2021, *arXiv:2101.00350*.

[4] S. Farrag and W. Alexan, "Secure 3D data hiding technique based on a mesh traversal algorithm," *Multimedia Tools Appl.*, vol. 79, nos. 39–40, pp. 29289–29303, Oct. 2020.
[5] M. Płachta, M. Krzemień, K. Szczypiorski, and A. Janicki, "Detection of image steganography using deep learning and ensemble classifiers," *Electronics*, vol. 11, no. 10, p. 1565, May 2022.
[6] M. A. Wani and B. Sultan, "Deep learning based image steganography: A review," *WIREs Data Mining Knowl. Discovery*, vol. 13, no. 3, p. 1481, May 2023.
[7] J. Zhang, X. He, and Y. Cao, "Errorless robust JPEG steganography using steganographic polar codes," *EURASIP J. Inf. Secur.*, vol. 2024, no. 1, p. 22, Jul. 2024.
[8] X. Duan, D. Guo, N. Liu, B. Li, M. Gou, and C. Qin, "A new high capacity image steganography method combined with image elliptic curve cryptography and deep neural network," *IEEE Access*, vol. 8, pp. 25777–25788, 2020.
[9] Y. Moussa and W. Alexan, "Message security through AES and LSB embedding in edge detected pixels of 3D images," in *Proc. 2nd Novel Intell. Lead. Emerg. Sci. Conf. (NILES)*, Oct. 2020, pp. 224–229.
[10] D. R. I. M. Setiadi, S. Rustad, P. N. Andono, and G. F. Shidik, "Digital image steganography survey and investigation (Goal, assessment, method, development, and dataset)," *Signal Process.*, vol. 206, May 2023, Art. no. 108908.
[11] M. H. Kombrink, Z. J. M. H. Geradts, and M. Worring, "Image steganography approaches and their detection strategies: A survey," *ACM Comput. Surveys*, vol. 57, no. 2, pp. 1–40, Feb. 2025.
[12] A. Kumar, R. Rani, and S. Singh, "A survey of recent advances in image steganography," *Secur. Privacy*, vol. 6, no. 3, p. 281, May 2023.
[13] W. Alexan, E. Mamdouh, A. Aboshousha, Y. S. Alsahafi, M. Gabr, and K. M. Hosny, "Stegocrypt: A robust tri-stage spatial steganography algorithm using TLM encryption and DNA coding for securing digital images," *IET Image Process.*, vol. 18, no. 13, pp. 4189–4206, Nov. 2024.
[14] D. R. I. M. Setiadi and J. Jumanto, "An enhanced LSB-image steganography using the hybrid Canny–Sobel edge detection," *Cybern. Inf. Technol.*, vol. 18, no. 2, pp. 74–88, Jun. 2018.
[15] I. Kich, E. B. Ameur, and Y. Taouil, "CNN auto-encoder network using dilated inception for image steganography," *Int. J. Fuzzy Log. Intell. Syst.*, vol. 21, no. 4, pp. 358–368, Dec. 2021.
[16] K. Gaurav and U. Ghanekar, "Image steganography based on Canny edge detection, dilation operator and hybrid coding," *J. Inf. Secur. Appl.*, vol. 41, pp. 41–51, Aug. 2018.
[17] L. Li, B. Luo, Q. Li, and X. Fang, "A color images steganography method by multiple embedding strategy based on Sobel operator," in *Proc. Int. Conf. Multimedia Inf. Netw. Secur.*, vol. 2, Nov. 2009, pp. 118–121.
[18] M. Alanzy, R. Alomrani, B. Alqarni, and S. Almutairi, "Image steganography using LSB and hybrid encryption algorithms," *Appl. Sci.*, vol. 13, no. 21, p. 11771, Oct. 2023.
[19] S. Chakraborty, A. S. Jalal, and C. Bhatnagar, "LSB based non blind predictive edge adaptive image steganography," *Multimedia Tools Appl.*, vol. 76, no. 6, pp. 7973–7987, Mar. 2017.
[20] K. Muhammad, J. Ahmad, N. U. Rehman, Z. Jan, and M. Sajjad, "CISSKA-LSB: Color image steganography using stego key-directed adaptive LSB substitution method," *Multimedia Tools Appl.*, vol. 76, no. 6, pp. 8597–8626, Mar. 2017.
[21] A. Malik, S. Singh, and R. Kumar, "Recovery based high capacity reversible data hiding scheme using even-odd embedding," *Multimedia Tools Appl.*, vol. 77, pp. 15803–15827, 2018.
[22] S. Kumar, A. Singh, and M. Kumar, "Information hiding with adaptive steganography based on novel fuzzy edge identification," *Defence Technol.*, vol. 15, no. 2, pp. 162–169, Apr. 2019.
[23] M. K. Shyla, K. B. Shiva Kumar, and R. K. Das, "Image steganography using genetic algorithm for cover image selection and embedding," *Soft Comput. Lett.*, vol. 3, Dec. 2021, Art. no. 100021.
[24] M. A. Ahmad, M. Elloumi, A. H. Samak, A. M. Al-Sharafi, A. Alqazzaz, M. A. Kaid, and C. Iliopoulos, "Hiding patients' medical reports using an enhanced wavelet steganography algorithm in DICOM images," *Alexandria Eng. J.*, vol. 61, no. 12, pp. 10577–10592, Dec. 2022.
[25] L. Zhu, X. Luo, C. Yang, Y. Zhang, and F. Liu, "Invariances of JPEG-quantized DCT coefficients and their application in robust image steganography," *Signal Process.*, vol. 183, Jun. 2021, Art. no. 108015.
[26] W. Ding, H. Zhang, R. Reulke, and Y. Wang, "Reversible image data hiding based on scalable difference expansion," *Pattern Recognit. Lett.*, vol. 159, pp. 116–124, Jul. 2022.
[27] S. A. Jebur, A. K. Nawar, L. E. Kadhim, and M. M. Jahefer, "Hiding information in digital images using LSB steganography technique," *Int. J. Interact. Mobile Technol.*, vol. 17, no. 7, pp. 167–178, Apr. 2023.
[28] A. Alabaichi, M. A. A. K. Al-Dabbas, and A. Salih, "Image steganography using least significant bit and secret map techniques," *Int. J. Electr. Comput. Eng. (IJECE)*, vol. 10, no. 1, p. 935, Feb. 2020.
[29] I. Kich, "Image steganography by deep CNN auto-encoder networks," *Int. J. Adv. Trends Comput. Sci. Eng.*, vol. 9, no. 4, pp. 4707–4716, Aug. 2020.
[30] M. Habiban, F. R. Hamade, and N. A. Mohsin, "Hybrid edge detection methods in image steganography for high embedding capacity," *Cybern. Inf. Technol.*, vol. 24, no. 1, pp. 157–170, Mar. 2024.
[31] A. U. Rehman, R. Rahim, M. S. Nadeem, and S. U. Hussain, "End-to-end trained CNN encoder–decoder networks for image steganography," in *Proc. Eur. Conf. Comput. Vis. (ECCV) workshops*, Jan. 2019, pp. 723–729.
[32] S. K. Ghosal, A. Chatterjee, and R. Sarkar, "Image steganography based on kirsch edge detection," *Multimedia Syst.*, vol. 27, no. 1, pp. 73–87, Feb. 2021.
[33] Z. Fouroozesh and J. Al Ja'am, "Image steganography based on LSBMR using Sobel edge detection," in *Proc. 3rd Int. Conf. e-Technol. Netw. Develop. (ICeND)*, Apr. 2014, pp. 141–145.
[34] O. R. Vincent and O. Folorunso, "A descriptive algorithm for Sobel image edge detection," in *Proc. InSITE Conf.*, 2009, pp. 97–107.
[35] A. J. Q. Almaliki, S. M. Abd, I. A. Lafta, R. Din, O. Ghazali, J. Q. Almaliki, and S. Utama, "Application of the Canny filter in digital steganography," *J. Adv. Res. Comput. Appl.*, vol. 35, no. 1, pp. 21–30, May 2024.
[36] W. McIlhagga, "The Canny edge detector revisited," *Int. J. Comput. Vis.*, vol. 91, no. 3, pp. 251–261, Feb. 2011.
[37] J. Bai, C.-C. Chang, T.-S. Nguyen, C. Zhu, and Y. Liu, "A high payload steganographic algorithm based on edge detection," *Displays*, vol. 46, pp. 42–51, Jan. 2017.
[38] A. M. Alattar, "Reversible watermark using the difference expansion of a generalized integer transform," *IEEE Trans. Image Process.*, vol. 13, no. 8, pp. 1147–1156, Aug. 2004.
[39] A. Sukumar, V. Subramaniyaswamy, L. Ravi, V. Vijayakumar, and V. Indragandhi, "Robust image steganography approach based on RIWT-Laplacian pyramid and histogram shifting using deep learning," *Multimedia Syst.*, vol. 27, no. 4, pp. 651–666, Aug. 2021.
[40] Y. Wang, M. Tang, and Z. Wang, "High-capacity adaptive steganography based on LSB and Hamming code," *Optik*, vol. 213, Jul. 2020, Art. no. 164685.
[41] S. K. Ghosal, J. K. Mandal, and R. Sarkar, "High payload image steganography based on Laplacian of Gaussian (LoG) edge detector," *Multimedia Tools Appl.*, vol. 77, no. 23, pp. 30403–30418, Dec. 2018.
[42] A. K. Sahu, G. Swain, M. Sahu, and J. Hemalatha, "Multi-directional block based PVD and modulus function image steganography to avoid FOBP and IEP," *J. Inf. Secur. Appl.*, vol. 58, May 2021, Art. no. 102808.
[43] S. Mukhopadhyay, S. Hossain, S. K. Ghosal, and R. Sarkar, "Secured image steganography based on Catalan transform," *Multimedia Tools Appl.*, vol. 80, no. 9, pp. 14495–14520, Apr. 2021.
[44] M. S. Taha, M. S. M. Rahem, M. M. Hashim, and H. N. Khalid, "High payload image steganography scheme with minimum distortion based on distinction grade value method," *Multimedia Tools Appl.*, vol. 81, no. 18, pp. 25913–25946, Jul. 2022.
[45] P. C. Mandal, I. Mukherjee, and B. N. Chatterji, "High capacity reversible and secured data hiding in images using interpolation and difference expansion technique," *Multimedia Tools Appl.*, vol. 80, no. 3, pp. 3623–3644, Jan. 2021.
[46] S. Rustad, I. M. S. De Rosal, P. N. Andono, A. Syukur, and P., "Optimization of cross diagonal pixel value differencing and modulus function steganography using edge area block patterns," *Cybern. Inf. Technol.*, vol. 22, no. 2, pp. 145–159, Jun. 2022.
[47] Y. Zhang, X. Luo, J. Wang, Y. Guo, and F. Liu, "Image robust adaptive steganography adapted to lossy channels in open social networks," *Inf. Sci.*, vol. 564, pp. 306–326, Jul. 2021.
[48] W. Lu, J. Zhang, X. Zhao, W. Zhang, and J. Huang, "Secure robust JPEG steganography based on AutoEncoder with adaptive BCH encoding," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 7, pp. 2909–2922, Jul. 2021.

[49] K. Zeng, K. Chen, W. Zhang, Y. Wang, and N. Yu, "Improving robust adaptive steganography via minimizing channel errors," *Signal Process.*, vol. 195, Jun. 2022, Art. no. 108498.
[50] D. Wang, G. Yang, J. Chen, and X. Ding, "GAN-based adaptive cost learning for enhanced image steganography security," *Expert Syst. Appl.*, vol. 249, Sep. 2024, Art. no. 123471.
[51] G. Xu, H.-Z. Wu, and Y.-Q. Shi, "Structural design of convolutional neural networks for steganalysis," *IEEE Signal Process. Lett.*, vol. 23, no. 5, pp. 708–712, May 2016.
[52] J. Ye, J. Ni, and Y. Yi, "Deep learning hierarchical representations for image steganalysis," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 11, pp. 2545–2557, Nov. 2017.



**RANA AL-RAWASHDEH** received the bachelor's degree in computer Science from Albalqa Applied University, Jordan, in 2009, and the master's degree in computer science from Jordan University of Science and Technology, Jordan, in 2019. Her research interests include machine learning, deep learning, cybersecurity, and data mining.



**MD MAHFUZUR RAHMAN** received the M.Sc. and Ph.D. degrees in computer science from the University of Manitoba, Canada. He is currently an Assistant Professor with the Department of Information and Computer Science (ICS), King Fahd University of Petroleum and Minerals (KFUPM), Saudi Arabia. He is also with the Interdisciplinary Research Center for Intelligent Secure Systems (IRC-ISS), KFUPM. His current research interests include cloud computing, cybersecurity, blockchain, and machine learning.



**MAHMOOD NIAZI** received the M.Phil. degree from The University of Manchester, Manchester, U.K., and the Ph.D. degree from the University of Technology Sydney, Sydney, NSW, Australia. He is currently a Professor of software engineering with the King Fahd University of Petroleum and Minerals, Dhahran, Saudi Arabia. Previously, he was with Keele University, Keele, U.K.; the National ICT Australia, Sydney; the University of Technology Sydney; The University of Sydney, Sydney; and The University of Manchester. He has over 20 years of international experience, eight years in the software industry, and more than 12 years in academia. He has published over 100 papers in peer-reviewed journals and conference proceedings. His research interests include sustainable, reliable, and secure software engineering processes, software security engineering, evidence-based software engineering, and insider-threats management.